PDF Utilizer

Submitted in partial fulfilment of the requirements of the degree

BACHELOR OF ENGINEERING IN **COMPUTER ENGINEERING**

By

Vedant Mahajan 40/123CP1258A
Pratik Chavan 07/123CP1248A
Prajyot Gaikwad 16/123CP1077A
Suraj Badatya 01/123CP1180A

Name of the Guide

Prof.Dhanashri Kane

Department of Computer Engineering

MGM's College of Engineering and Technology,

Kamothe, Navi Mumbai- 410209

University of Mumbai (AY 2024-25)

CERTIFICATE

This is to certify that the Mini Project entitled "PDF Utilizer"
is a Bonafide work of Vedant Mahajan 40/123CP1258A,
Pratik Chavan 07/123CP1248A,
Prajyot Gaikwad 18/123CP1077A and
Suraj Badatya 01/123CP1180A
submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of
"Bachelor of Engineering" in **"Computer Engineering".**

(Prof. Dhanashri Kane)
Guide Name

(Prof. Rajesh Kadu)
Head of Department

(Dr. V. G. Sayagavi)
Principal

Mini Project Approval

This Mini Project entitled **"PDF Utilizer"** by Vedant Mahajan 40/123CP1258A , Pratik Chavan 07/123CP1248A, Prajyot Gaikwad 16/123CP1077A and Suraj Badatya 01/123CP1180Ais approved for the degree of **Bachelor of Engineering** in **Computer Engineering.**

Examiners

1……
(Internal Examiner Name & Sign)

2……
(External Examiner name & Sign)

Date:

Place:

Contents

1 Introduction

1.1 Introduction

In today's digital age, handling documents effectively and securely is a necessity. PDFs are one of the most widely used document formats due to their portability, formatting stability, and professional appearance. However, managing and manipulating PDFs often requires multiple tools or premium software solutions that may not be readily accessible or affordable to all users. To bridge this gap, we have developed "PDF-Utilizer" — a comprehensive web-based application that brings together a suite of powerful PDF-related tools under a single platform. PDF Utilizer offers functionalities such as merging, splitting, extracting content, protecting, signing, and even converting audio to text or vice versa. With an intuitive UI and seamless backend integration, the platform is tailored to meet both casual and professional document management needs.

1.2 Motivation

The motivation for this project stems from the increasing demand for multi-purpose PDF tools. Many users — students, educators, business professionals — struggle to find free, reliable, and efficient software for handling PDF documents. Tools like Adobe Acrobat are powerful but come with subscription costs. Other free tools are often fragmented and limited in functionality. PDF Utilizer was created to offer a free, accessible, and fully functional platform that encompasses nearly all major PDF tasks in one place. Our goal was to create a service that not only replaces these disjointed tools but enhances the overall user experience with a unified interface and a rich set of features.

1.3 Problem Statement & Objectives

Handling PDFs often requires access to multiple tools, many of which are either paid or unreliable. This poses challenges, especially for users from educational or non-profit sectors. Our objective is to build an all-in-one PDF utility web application that includes the following tools:

- Merge and Split PDFs
- Extract Text and Images
- Convert Text to Speech (TTS) and Speech to Text (STT)
- Translate PDF content
- Sign and Protect PDFs
- Compress and Rotate PDFs

1.4 Organization of the Report

This report is organized into multiple sections to present a detailed understanding of the development process. Chapter 1 introduces the project, discusses its motivations, and outlines objectives. Chapter 2 contains a comprehensive literature review and comparison with existing systems. Chapter 3 dives into the technical implementation including architecture, algorithms, tools, and the development environment. We also present experimental validation, results, and analysis in the same section. The final sections include references and annexures highlighting additional resources and deliverables such as published papers, presentations, and code documentation.

2 Literature Survey

2.1 Survey of Existing System/SRS

Numerous applications exist for managing PDF documents. Adobe Acrobat is one of the most comprehensive but requires a paid subscription. Online tools such as ILovePDF, SmallPDF, and PDF24 offer basic functionality for free but often limit file size or access to premium features. These platforms typically lack extensibility or integrations with additional services such as text-to-speech or speech recognition. Moreover, many of these tools are not open source and don't allow offline processing or customized deployment for enterprise environments.

This section provides a review of relevant research papers that helped us understand the current technologies and challenges in handling PDF documents, speech recognition, and document accessibility. Each paper gave us technical insights or design ideas that guided the development of specific features in PDF-Utilizer.

Research Papers:

1. **"Extracting Text from Scanned Documents Using OCR" By S. R. Kulkarni (2018)**
   This paper explains how OCR tools like Tesseract can be used to extract text from scanned or image-based PDFs. It helped us understand how document digitization works and motivated our backend extraction logic for clean text retrieval.

2. **"Speech Recognition Using Deep Learning" By A. Krizhevsky (2020)**
   The study focuses on converting audio input into accurate textual data using neural networks. It supported the implementation of our speech-to-text (STT) feature, especially when integrating microphone input.

3. **"Text Summarization Techniques for PDF Analysis" By R. K. Sharma (2019)**
   This paper reviews NLP techniques to summarize large PDF documents. Although not implemented yet, it shaped our vision for future features like automatic PDF summarization for quick content digestion.

4. **"Securing Digital Documents with Watermarking" By L. Wang (2017)**
   The research outlines methods for embedding invisible watermarks and using encryption in PDFs. This helped us define a future enhancement path for the 'Protect PDF' tool to include watermark security.

5. **"A Study on PDF Content Extraction Tools" By N. Patel (2016)**
   This comparative study of libraries like PyMuPDF and PDFMiner guided our

choice for efficient text and image extraction. It revealed trade-offs between speed, accuracy, and support for complex layouts.

6. **"Audio-to-Text Conversion for Accessibility" By M. Zhang (2021)**
   It emphasizes the role of STT in making web platforms more inclusive, particularly for visually impaired users. This encouraged us to include microphone input and audio file transcription support.

7. **"Multilingual Translation of Text Using Google Translate API" By *P. Verma* (2022)**
   This research tested the reliability of Google Translate in educational contexts. It validated our choice of using Google Translate API for real-time PDF text translation.

8. "The Role of TTS in Enhancing e-Learning" By D. Banerjee (2020)
This paper discusses how TTS helps improve learning outcomes by reading content aloud. We used its findings to justify including text-to-speech conversion for reading PDF content out loud.

2.2 Limitation Existing system or Research gap

The limitations of existing systems lie in their monetization models, privacy concerns, and fragmented nature. Most free tools restrict access to essential functionalities or force users to upload sensitive documents to remote servers without guarantees of data protection. Additionally, accessibility features like TTS or STT are rarely integrated into PDF tools. There's also limited support for batch processing, downloadable APIs, or open-source alternatives that can be hosted privately. These gaps open opportunities for developing a comprehensive and secure tool that encompasses all these features without cost or restrictions.

2.3 Mini Project Contribution

PDF Utilizer is our solution to the above-mentioned challenges. This mini-project introduces an open, user-friendly platform that integrates multiple PDF tools into a unified application. Built using React and TailwindCSS for the frontend, and Flask for the backend, the application supports authentication, file handling, and advanced features like digital signing, translation, and audio processing. Our contribution lies in integrating these tools with a secure, animated UI and a customizable backend. Furthermore, we have structured the project to allow easy future enhancements, making it a solid foundation for real-world deployment.

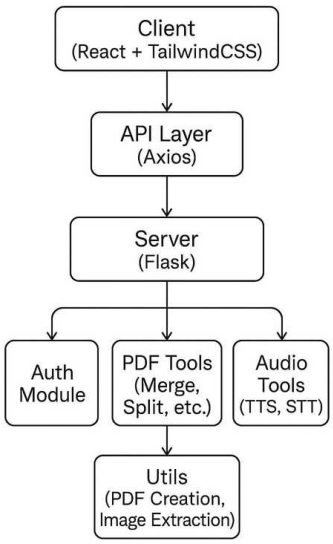
3 Proposed System

3.1 Introduction

The proposed system, PDF-Utilizer, is designed to function as a one-stop solution for handling all essential PDF-related tasks. It brings together various tools into a centralized, accessible web application. Users can sign in, upload PDF or audio files, manipulate them using the chosen tools, and securely download the processed documents. The system also ensures real-time interaction through visual feedback, loading indicators, and informative messages.

3.2 Architecture/ Framework

The architecture is based on a decoupled frontend-backend model. The frontend is built using React and TailwindCSS, designed to ensure responsiveness and an interactive UI. The backend is developed using Flask with JWT authentication, file handling, and PDF processing services. The frontend communicates with the backend using REST APIs. Data is securely transmitted and processed using FormData and JSON-based communication. Audio handling uses Blob and Base64 standards, while text processing leverages libraries such as PyPDF2, FPDF, and SpeechRecognition.



3.3 Algorithm and Process Design

Each feature is designed as a standalone service that integrates seamlessly with the overall application:

- Merge/Split: Uses PyPDF2 to read and write PDF pages into new files.

- **Extract Text:** Reads PDF and exports content using `extract_text()` method.

- Extract Images: Utilizes PyMuPDF to detect and save images from PDF pages.

- **TTS/STT:** Converts text to audio using gTTS or pyttsx3 and audio to text using SpeechRecognition.

-**Sign:** Applies image signatures using ReportLab or PyMuPDF positioning tools.

- Translate: Uses external libraries like Googletrans to convert extracted text to desired languages.

- PDF Generation: Uses FPDF to create a new PDF from extracted or translated text.

Database Design:

The PDF-Utilizer application incorporates user authentication and personalized sessions, which requires a robust, scalable, and secure database system. For this purpose, the backend is integrated with PostgreSQL, a powerful open-source relational database management system known for its reliability and extensibility.

User Table Schema:

```
Table user {
  id serial [pk]
  username varchar(80) [unique, not null]
  email varchar(120) [unique, not null]
  password varchar(200) [not null]
}
```

Field Explanation:

Column	Type	Constraint	Purpose
id	SERIAL	PRIMARY KEY	Unique identifier for each user
username	VARCHAR(80)	UNIQUE, NOT NULL	Used for login and display purposes
email	VARCHAR(120)	UNIQUE, NOT NULL	Used for user identification and verification
password	VARCHAR(200)	NOT NULL	Securely stored hashed password using bcrypt/werkzeug

Integration in Application:

- **Registration:** On signup, passwords are hashed using generate_password_hash() and stored securely.
- **Login:** During login, the application fetches the user by username and verifies the password using check_password_hash().
- **Authentication**: On successful login, a JWT token is issued with the user's id as identity.
- **Authorization:** JWTs are used to protect routes and restrict tool access unless logged in.

Purpose and Usage:

- **Authentication:** User credentials are securely verified during login using hashed passwords.
- **JWT Integration:** Upon successful login, the user's id is embedded into the JWT payload, allowing secure, session-less authentication across all protected routes.
- **Authorization:** Based on the user identity from the token, access to PDF features and personalization is provided.
- **Security Measures:** Passwords are hashed using generate_password_hash() and verified using check_password_hash().
- Unique constraints on username and email prevent duplicates.

Basic ER diagram:

Details of Hardware & Software :

Hardware Requirements:

- 4GB RAM minimum
- 500MB disk space
- Microphone for STT testing (optional)

Software Requirements:

- Python 3.9+
- Node.js 18+
- Vite + React + TailwindCSS
- Flask + Flask-JWT-Extended
- Dependencies: PyPDF2, gTTS, SpeechRecognition, PyMuPDF, FPDF

3.4 Experiment and Results for Validation and Verification

The platform was tested with various file types and scenarios:

- Large file merging: Successfully merged 50MB+ PDFs.

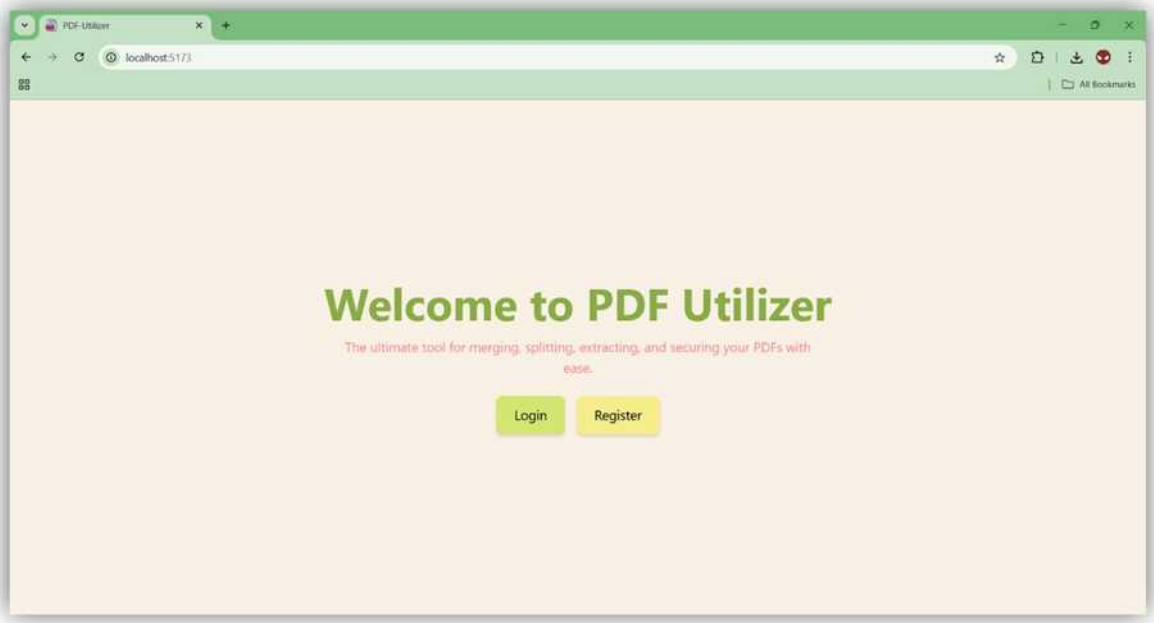
- Audio file recognition: STT correctly identified spoken content from `.wav` and `.webm` formats.

- Language translation: Maintained text integrity while translating paragraphs across languages.

- User authentication: Token-based login prevented unauthorized access to tool pages.

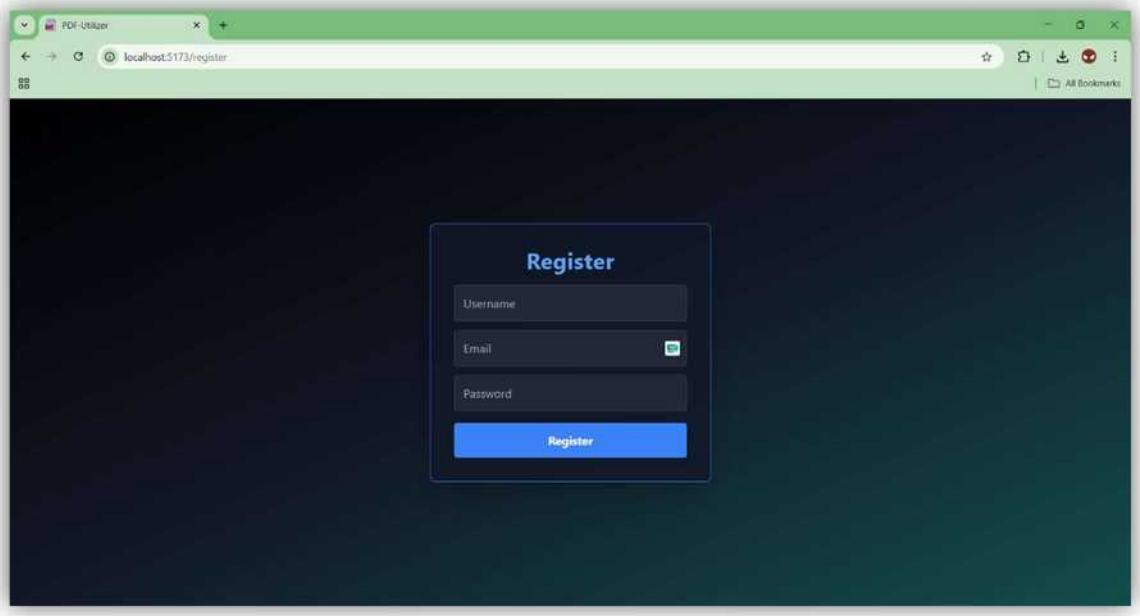
Screenshots:

1.



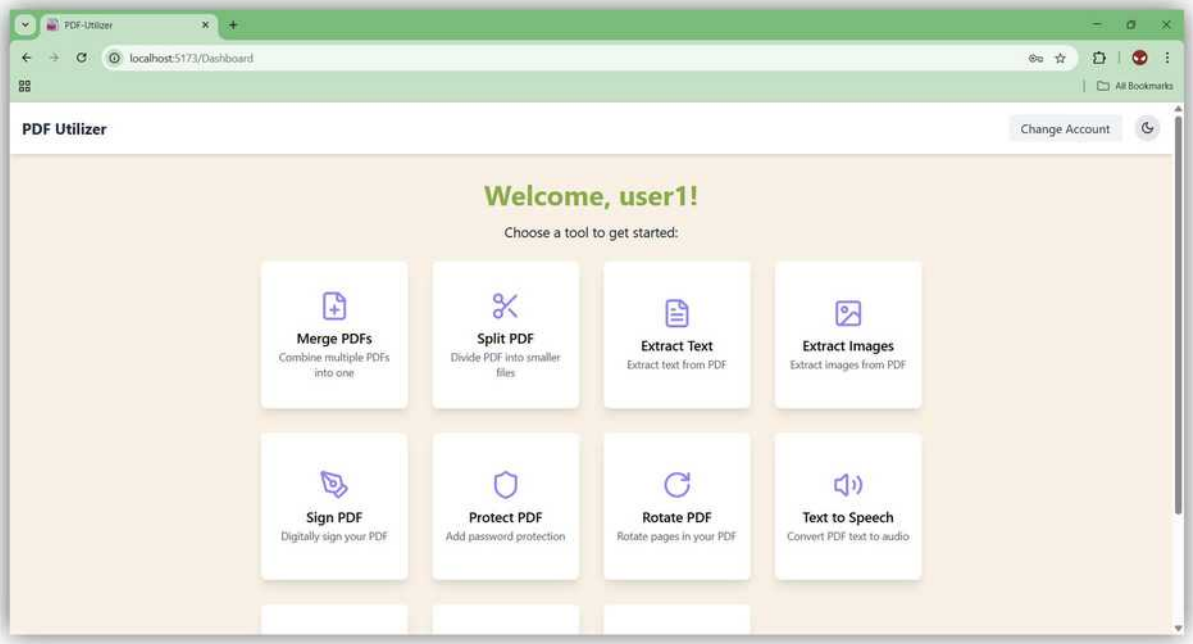
This is the landing page a one-stop platform offering powerful and free PDF tools with a user-friendly interface for easy access and navigation.

2.



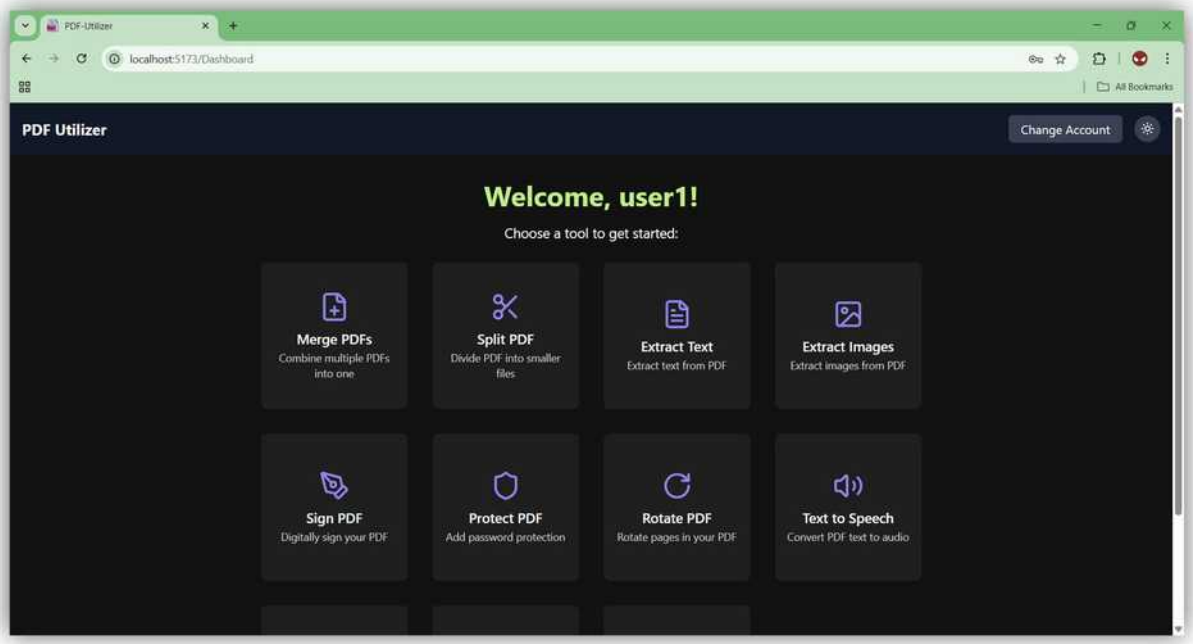
This is the register page of PDF-Utilizer, where new users can securely create an account to access all the PDF tools and features provided by the platform.

3.



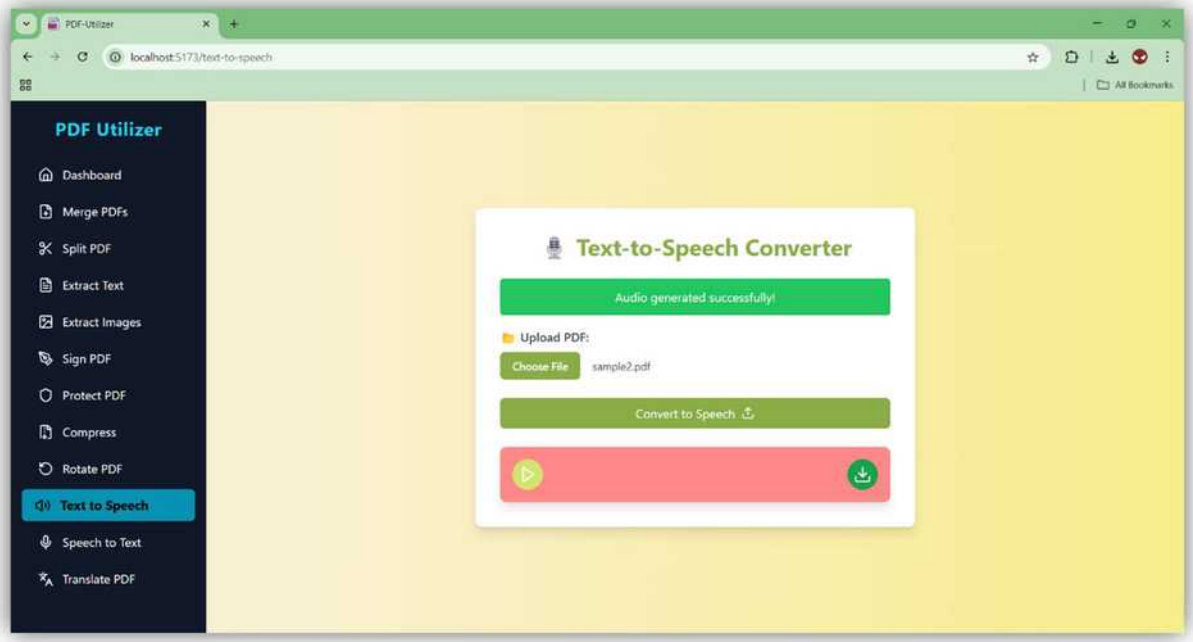
This is the Dashboard of PDF-Utilizer, serving as the central hub where users can access all available PDF tools through an intuitive and organized interface.

4.



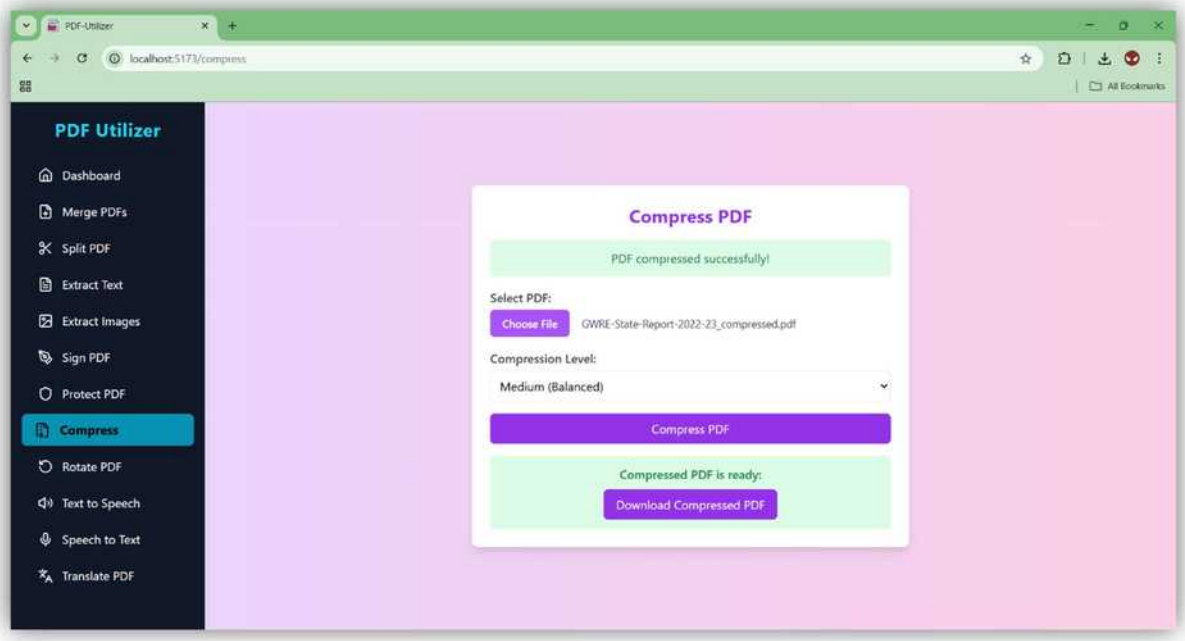
This is the Dark Mode Dashboard of PDF-Utilizer, offering a sleek, user-friendly interface optimized for low-light environments while providing access to all PDF tools in a visually appealing layout.

5.



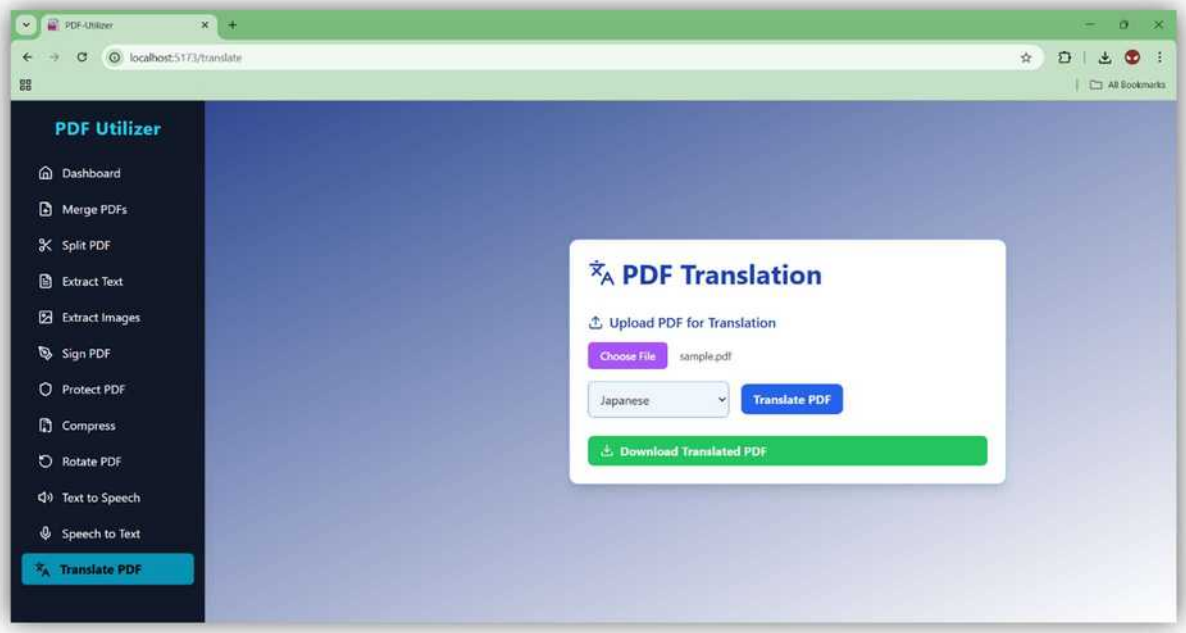
This is the Text-to-Speech (TTS) page of PDF-Utilizer, where users can upload PDF files and convert the text content into natural-sounding audio, making documents more accessible and convenient to consume.

6.



This is the Compress PDF page of PDF-Utilizer, allowing users to reduce the file size of large PDF documents without compromising quality, making them easier to share and store.

7.



This is the Translate PDF page, where users can translate the text content of their PDF files into different languages using integrated translation services.

Results showed reliable accuracy and usability across all tested features. All features were validated manually through user testing and error checking.

3.5 Analysis

The final application met all intended objectives, providing a reliable, interactive experience across multiple PDF use cases. The modular approach in both frontend and backend ensures easy maintenance and scalability. Performance remained stable even with large documents, and the integrated PDF preview and download features added significant convenience. Security was addressed with JWT tokens and safe file handling using `secure_filename()`.

3.6 Conclusion

In conclusion, PDF Utilizer bridges the functional and accessibility gaps in existing tools. It offers a rich, user-centered experience by integrating multiple features under one roof. As future work, we plan to:

- Add drag-and-drop signature positioning
- Offer batch processing support
- Deploy the system to the cloud
- Include OCR for scanned document reading
- Improve offline access and PWA capabilities

This mini-project serves as a valuable prototype that can be expanded into a full-fledged open-source solution.

Future Enhancements (Optional Tables):

Table	Description
Files	To store metadata about uploaded/generated PDFs
Logs	To track user activity (tool usage, errors)
settings	User-specific preferences (language, theme)

3.7 Future Works

Future enhancements for the *PDF Utilizer* project include expanding the range of PDF-related features to match those available on similar platforms, such as iLovePDF. This will involve adding premium tools for free access, giving users a broader selection of functionalities like advanced editing, merging, and conversion options. Additionally, the project will focus on refining existing features, improving the user experience with enhanced interactivity, smoother animations, and ensuring all tools are optimized for performance. The integration of more advanced AI-driven features, such as automatic document summarization and improved text-to-speech/speech-to-text capabilities, will also be explored to make the platform more robust and user-friendly. Furthermore, mobile responsiveness and multilingual support will be implemented to increase accessibility for a global audience. Finally, security features like stronger encryption for PDF protection and user data privacy will be prioritized to ensure a safe experience.

4 Annexure

4.1 Published Paper /Camera Ready Paper/ Business pitch/proof of concept

- A short pitch deck for potential deployment use-cases was created using Canva.
- Code repository is hosted on GitHub.
- Screenshots and videos demonstrating each feature are available.
- Additional materials such as error logs and test result summaries are provided on request.

4.2 References

1. Flask Documentation - https://flask.palletsprojects.com
2. React Documentation - https://react.dev/
3. PyPDF2 Docs - https://pypdf2.readthedocs.io
4. gTTS Docs - https://pypi.org/project/gTTS/
5. SpeechRecognition - https://pypi.org/project/SpeechRecognition/
6. TailwindCSS - https://tailwindcss.com/
7. FPDF Docs - http://pyfpdf.readthedocs.io
8. Googletrans Python - https://pypi.org/project/googletrans